CI/CD for Node.js Retail App

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	ANGEL GURJAR	EN22CS301123
2.	SAHIL SHRICHAND TALREJA	EN22CS301838
3.	KIRTI CHOUDHARY	EN22CS301508
4.	PAVANI KANOONGO	EN22CS301699
5.	MAHI VISHWAKARMA	EN22CS301568
6.	SAIYAD ISHAN ALI	EN22CS301839

Group Name:G-01

Project Number:DO-01

Industry Mentor Name:

University Mentor Name:

Academic Year:2022-2026

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used (*If applicable)*

Results & Output

Conclusion

Future Scope & Enhancements

1. Problem Statement & Objectives

1.1. Problem Statement

Manual software deployment in retail environments leads to "Human Error," inconsistent environments, and slow release cycles. A developer might forget to run tests or use a different version of Node.js than the server, causing production outages.

1.2. Project Objectives

- To eliminate manual deployment steps through **Automation**.
- To ensure high code quality by enforcing **Automated Unit Testing**.
- To achieve environment consistency using **Docker Containerization**.

1.3. Scope of the Project

The scope covers the CI/CD pipeline from the moment code is pushed to GitHub, through the testing phase, ending with the creation of a verified Docker image.

---

2. Proposed Solution

2.1. Key Features

- **Trigger-based Automation:** Automatic execution on every code push.
- **Quality Gates:** Tests must pass for the build to proceed.
- **Immutable Artifacts:** Every build creates a unique Docker container.
- **Failure Notifications:** Immediate feedback to developers if a build fails.

2.2. Overall Architecture / Workflow

The workflow follows a linear "Commit-Test-Build" logic.

2.3. Tools & Technologies Used

- **Language:** Node.js (Runtime)
- **Version Control:** GitHub
- **CI/CD Engine:** GitHub Actions
- **Containerization:** Docker
- **Environment:** GitHub Codespaces (Development)

3. Results & Output

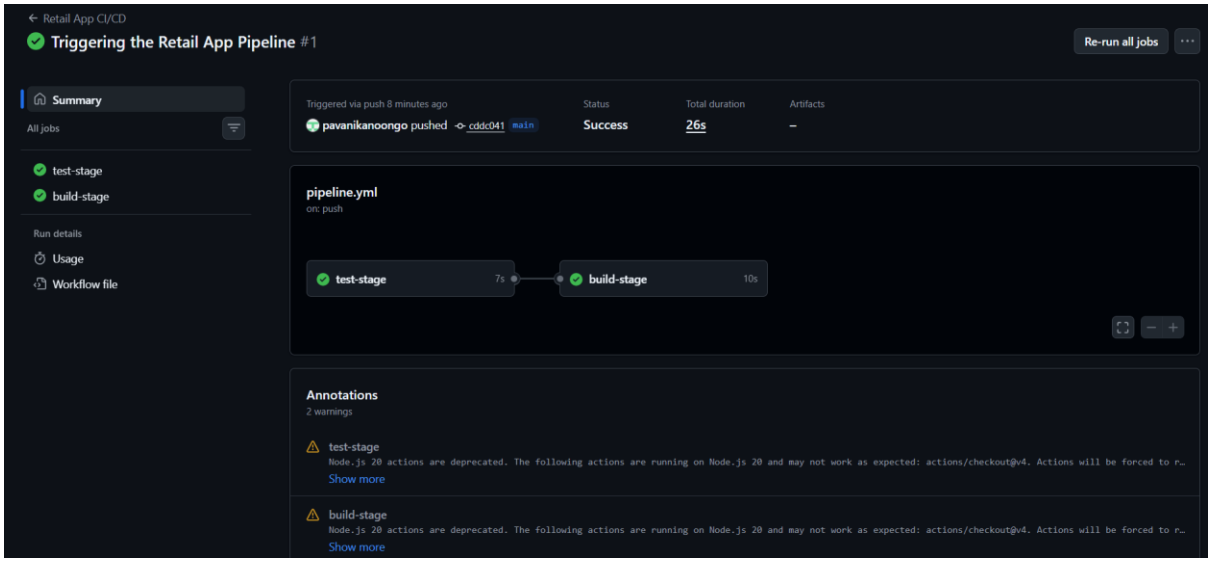
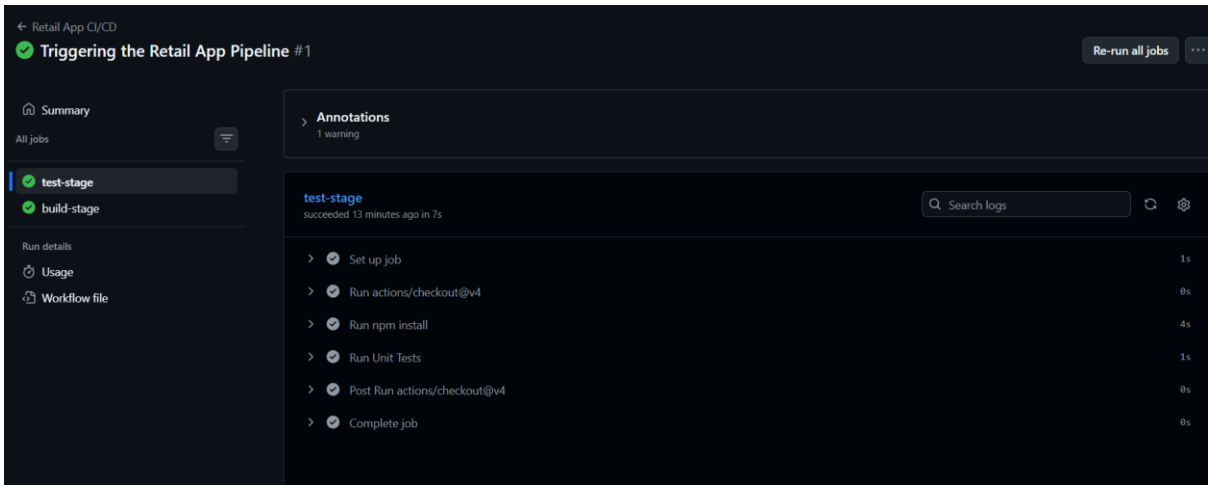
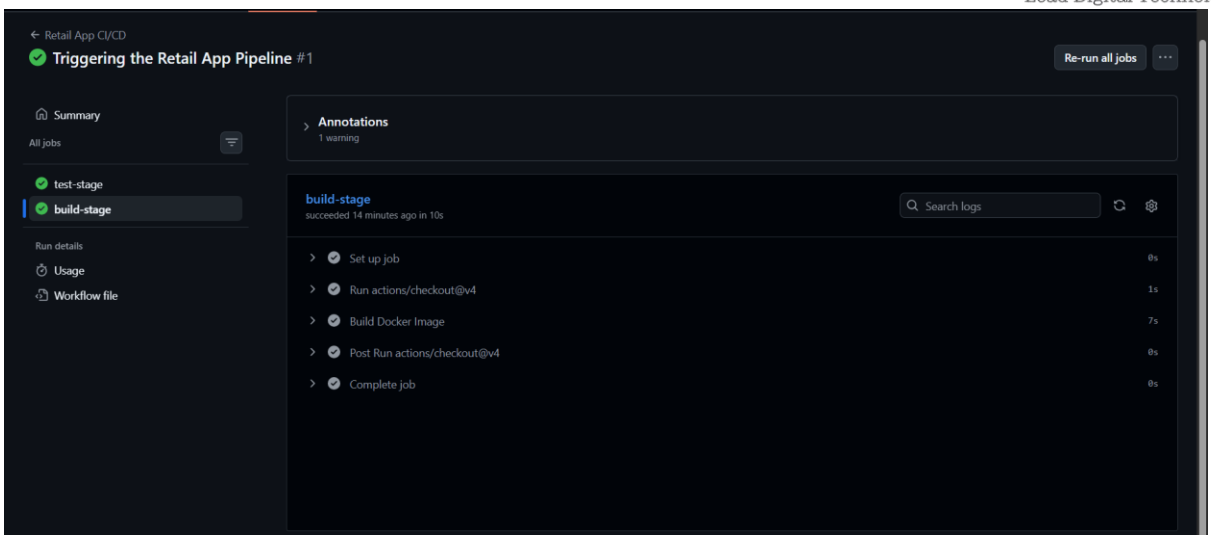
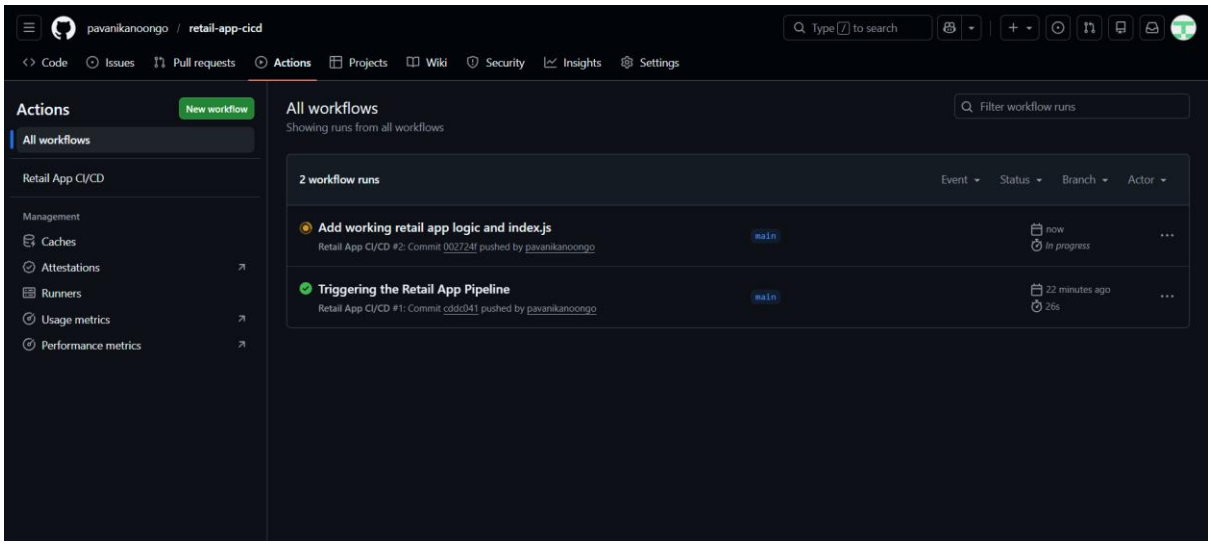
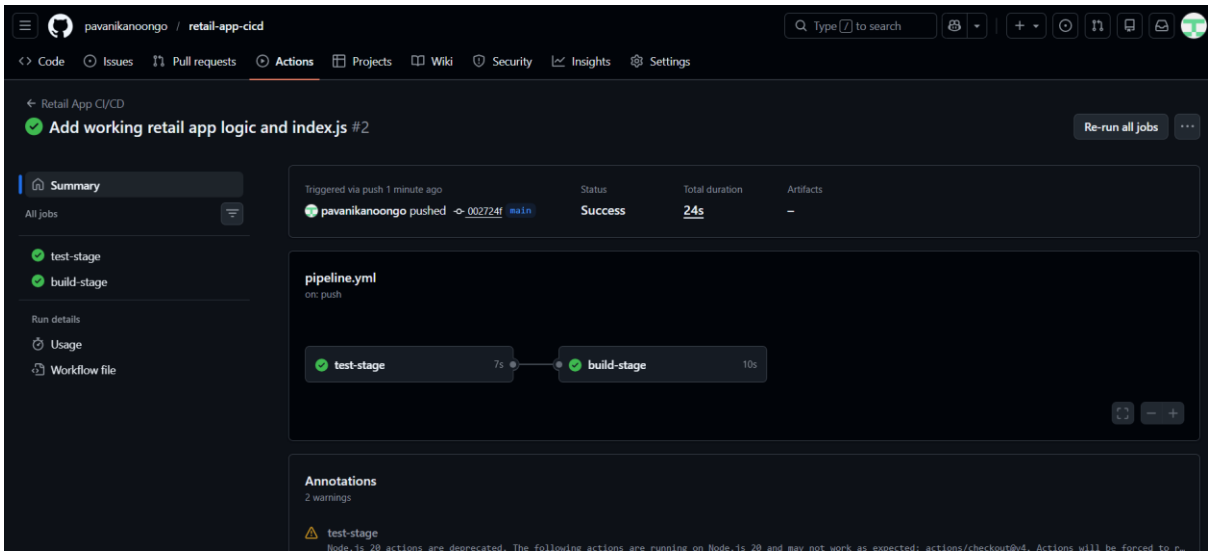


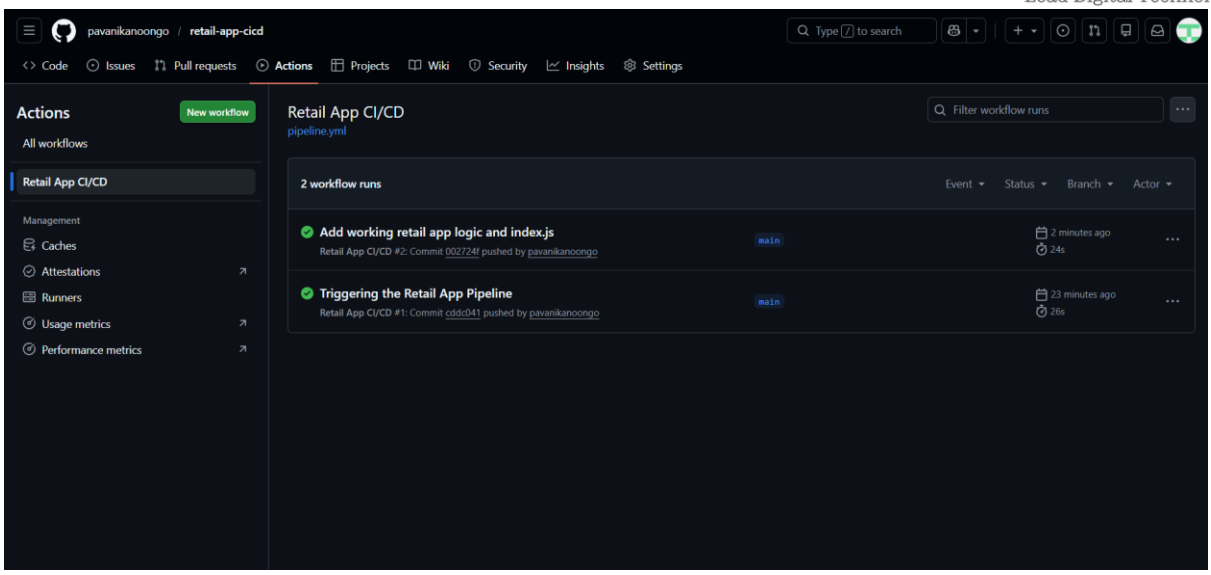
Fig.1 Automated CI/CD Workflow Execution




Retail App CI/CD
Triggering the Retail App Pipeline #1
Re-run all jobs
Summary
All jobs
test-stage
build-stage
Run details
Usage
Workflow file
Annotations
1 warning
build-stage
succeeded 14 minutes ago in 10s
Search logs
Set up job
Run actions/checkout@v4
Build Docker Image
Post Run actions/checkout@v4
Complete job


pavanikanoongo / retail-app-cicd
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Actions
New workflow
All workflows
Retail App CI/CD
Management
Caches
Attestations
Runners
Usage metrics
Performance metrics
All workflows
Showing runs from all workflows
Filter workflow runs
2 workflow runs
Event
Status
Branch
Actor
Add working retail app logic and index.js
Retail App CI/CD #2: Commit 002724f pushed by pavanikanoongo
main
now
In progress
Triggering the Retail App Pipeline
Retail App CI/CD #1: Commit cddc041 pushed by pavanikanoongo
main
22 minutes ago
26s


pavanikanoongo / retail-app-cicd
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Retail App CI/CD
Add working retail app logic and index.js #2
Re-run all jobs
Summary
All jobs
test-stage
build-stage
Run details
Usage
Workflow file
Triggered via push 1 minute ago
pavanikanoongo pushed 002724f main
Status
Success
Total duration
24s
Artifacts
–
pipeline.yml
on: push
test-stage
7s
build-stage
10s
Annotations
2 warnings
test-stage



4. Conclusion

This project successfully demonstrates a modern DevOps approach to software delivery. By integrating GitHub Actions and Docker, we transformed a standard Node.js application into a portable, enterprise-ready service.

Key Learning: The most critical part of a professional environment isn't just the code itself, but the **automated gates** that ensure only high-quality, tested code reaches the end user.

---

5. Future Scope & Enhancements

- **SAST Integration:** Adding security scanning (Snyk or npm audit) directly into the pipeline.
- **Auto-Deployment:** Extending the pipeline to deploy the Docker image automatically to a Kubernetes cluster.
- **Monitoring:** Integrating Prometheus/Grafana to track the health of the retail app after deployment.